

Class name: flashcards	
Class Attributes:	Class Collaborations (other classes):
- term (str): the word or phrase being studied - definition (str): the explanation of the term	
Class Methods:	Class Collaborations (other classes):
<code>__init__(self, term, definition)</code> Initializes a Flashcard object with a given term and definition. <code>__str__(self)</code> Returns the string representation of the flashcard (usually the term), useful for debugging or displaying.	

Class name: FlashcardAppGUI**Class Attributes:**

- manager (FlashcardManager):
Provides access to the flashcard data and handles navigation logic.
- GUI widgets (like term_label, definition_label, buttons):
Visual elements in the window that show the flashcard term, display the definition, and allow user interaction (buttons for flipping, next, etc.).

Class Collaborations (other classes):

-

Class Methods:

- __init__(self, manager)
Sets up the GUI window and links it with the flashcard manager. Creates all interface components (labels, buttons, etc.).
- show_term(self)
Displays the current flashcard's term on the screen.
- show_definition(self)
Reveals the current flashcard's definition.
- show_next_card(self)
Moves to the next flashcard and updates the GUI with the new term. If there are no more cards, you might show a message like "End of deck."

Class Collaborations (other classes):

-

Class name: FlashcardManager

Class Attributes:

- flashcards (list of Flashcard):
Stores all flashcards loaded from the .txt file.
- current_index (int):
Tracks the index of the current flashcard in the list. Starts at 0 and increases as the user moves forward.

Class Collaborations (other classes):

Class Methods:

- load_from_file(self, filename)
Reads the .txt file line by line, splitting each line into a term:definition pair. Each pair becomes a Flashcard object stored in the flashcards list.
- get_current_flashcard(self)
Returns the flashcard currently indexed by current_index.
- next_flashcard(self)
Advances current_index to the next card. If the end of the list is reached, it may loop back or stop based on how you implement it.
- has_next(self)
Returns True if there are more cards after the current one, else False.

Class Collaborations (other classes):